

Mini PROJECT PRESENTATION

INTELLIGENT MOBILE ROBOT
NAVIGATION USING COMPUTATIONAL
INTELLIGENCE TECHNIQUES

HYBRID OF FUZZY-GA-ANN

OUR TEAM

- NUR HUSNA ELYSA MAISARAH BINTI ROSLI (2310366)
- AUFA SIDQY BINTI MOHD SIDQY (2310542)
- FARHA QAISARA BINTI MOHD ZAHIDI (2315994)
- AMALIN BALQIS BINTI RUYUSNI (2314684)

introduction

1. What is Intelligent Mobile Robot?

An intelligent mobile robot is an autonomous system capable of navigating within an environment without continuous human control. It uses sensors to perceive the surroundings, processes the data using decision-making algorithms, and then executes appropriate movements. Unlike traditional robots that follow fixed instructions, intelligent robots can adapt their behavior based on environmental conditions, making them suitable for applications such as indoor service robots, automated delivery systems, and search-and-rescue operations.

INTRODUCTION

2. Obstacle Avoidance and Goal Reaching using Soft Computing

Mobile robot navigation becomes challenging when the environment contains obstacles and sensor data is uncertain.

Soft computing techniques such as Fuzzy Logic, Genetic Algorithms, and Neural Networks provide flexible solutions that can handle uncertainty. Fuzzy Logic helps the robot make smooth decisions using rule-based reasoning, Genetic Algorithms optimize controller parameters, and Neural Networks enable learning and adaptation. By combining these techniques, the robot can avoid obstacles and reach the goal efficiently in complex environments.

OBJECTIVES

- Develop a hybrid navigation controller using Fuzzy Logic + Genetic Algorithm + Neural Network.
- Enable the robot to move toward the goal while avoiding obstacles.
- Optimize navigation performance for smoother and shorter paths.
- Test and evaluate the system in different simulated environments.
- Show the benefits of hybrid AI methods compared to single techniques.

METHODOLOGY

WHY HYBRID ?!

- To address the limitations of using a single soft computing technique for mobile robot navigation
- Fuzzy logic provides interpretable, human-like reasoning and handles uncertainty effectively
- Genetic Algorithms are incorporated to systematically optimize navigation-related parameters
- Artificial Neural Networks introduce learning capability
- Delivers more efficient, smoother, and robust navigation performance, particularly in environments with high complexity.

FUZZY

Rule	IF	THEN
R1	Obstacle Distance is Very Near	Movement Direction is Blocked
R2	Obstacle Distance is Near	Movement Direction is Strongly Penalized
R3	Obstacle Distance is Medium	Movement Direction is Mildly Penalized
R4	Obstacle Distance is Far	Movement Direction is Not Penalized

Table 1. Each rule evaluates the suitability of a candidate movement direction based on the corresponding obstacle distance.

FUZZY

```
python Copy Download

# Lines 284-378 - Fuzzy Logic implementation
class FuzzyController:
    """Advanced Fuzzy Logic Controller"""
    def __init__(self, obstacle_w=0.8, goal_w=0.9, smooth_w=0.6):
        self.obstacle_w = obstacle_w
        self.goal_w = goal_w
        self.smooth_w = smooth_w

    def fuzzy_rules(self, controller_inputs, prev_dir, visited_positions
=None, grid=None):
        # Main fuzzy inference system with rule evaluation
        scores = []
        for i, (dx, dy) in enumerate(directions):
            # Rule 1: Obstacle distance scoring
            if obstacle_dist_this_dir < 0.8:
                scores.append(-10000)
            elif obstacle_dist_this_dir < 1.5:
                obs_score = -10.0 * (1.5 - obstacle_dist_this_dir)

            # Rule 2: Goal direction scoring
            goal_score = (1.0 - angle_diff / np.pi) * self.goal_w * 5.0

            # Rule 3: Path smoothness scoring
            smooth = 0
            if prev_dir:
                if (dx, dy) == prev_dir:
                    smooth = 2.0 * self.smooth_w

            # Rule 4: Exploration/avoid revisiting
            visit_penalty = 0
            if visited_positions and (nx, ny) in visited_positions:
                visit_penalty = -5.0 * visit_count

            # Combine all fuzzy rules
            total = (goal_score + obs_score + progress + smooth +
                    visit_penalty + exploration + left_bonus + right_bon
                    us +
                    up_bonus + down_bonus)
            scores.append(total)
        return scores, directions
```

```
# Rule 4: Exploration/avoid revisiting
visit_penalty = 0
if visited_positions and (nx, ny) in visited_positions:
    visit_penalty = -5.0 * visit_count

# Combine all fuzzy rules
total = (goal_score + obs_score + progress + smooth +
        visit_penalty + exploration + left_bonus + right_bon
        us +
        up_bonus + down_bonus)
scores.append(total)
return scores, directions
```


GENETIC ALGORITHM

Parameter	Value
Population Size	15 individuals
Number of Generations	8
Total Fitness Evaluations	120 simulations
Selection Method	Elite selection (top 4 individuals)
Crossover Method	Arithmetic crossover (average of two parents)
Mutation Method	Gaussian mutation (probability = 0.3, $\sigma = 0.1$)
Elitism Strategy	Best 4 individuals preserved

Table 2. Genetic Algorithm (GA) Configuration Parameter

GENETIC ALGORITHM

python

Copy Download

```
# Lines 519-628 - GA implementation
class GeneticOptimizer:
    """GA for parameter optimization"""
    def __init__(self, pop_size=20, gens=10):
        self.pop_size = pop_size
        self.gens = gens

    def create(self):
        # Create random parameters (chromosome)
        return {
            'obstacle': random.uniform(0.5, 1.0),
            'goal': random.uniform(0.7, 1.0),
            'smooth': random.uniform(0.4, 0.8)
        }

    def fitness(self, params, start, goal, grid):
        # Evaluate fitness score for each chromosome
        # ... fitness calculation code ...
```

```
def optimize(self, start, goal, grid):
    # Main GA loop with selection, crossover, mutation
    pop = [self.create() for _ in range(self.pop_size)]

    for gen in range(self.gens):
        fits = [self.fitness(ind, start, goal, grid) for ind in pop]
        # Selection: Keep best individuals (elite)
        elite = sorted_pop[:4]
        # Crossover and mutation
        while len(offspring) < self.pop_size - 4:
            p1, p2 = random.sample(elite + sorted_pop[:8], 2)
            child = {k: (p1[k]+p2[k])/2 for k in p1.keys()} # Cross
over
            if random.random() < 0.3: # Mutation probability
                k = random.choice(list(child.keys()))
                child[k] += random.gauss(0, 0.1) # Mutation
    return best # Return optimized parameters
```


NEURAL NETWORK

Parameter	Description
Network Type	Feedforward Neural Network
Input Parameters	Obstacle distances and goal orientation
Output	Movement direction refinement
Role in Controller	Refinement of fuzzy logic decision
Training Data	Simulation-generated navigation data
Learning Mode	Offline supervised learning
Integration Method	Combined with fuzzy logic controller

Table 3. Artificial Neural Network (ANN) Configuration

NEURAL NETWORK

```
python Copy Download
# Lines 179-245 - Neural Network implementation
class NeuralNetwork:
    """Neural Network for path prediction"""
    def __init__(self):
        # 3-layer neural network with ReLU activation
        self.W1 = np.random.randn(10, 32) * np.sqrt(2.0 / 10) # Input
        # Hidden1
        self.b1 = np.zeros((1, 32))
        self.W2 = np.random.randn(32, 16) * np.sqrt(2.0 / 32) # Hidden
        # Hidden2
        self.b2 = np.zeros((1, 16))
        self.W3 = np.random.randn(16, 8) * np.sqrt(2.0 / 16) # Hidden
        # Output
        self.b3 = np.zeros((1, 8))

    def forward(self, X):
        # Forward propagation
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = self.relu(self.z1) # ReLU activation

        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = self.relu(self.z2)

        self.z3 = np.dot(self.a2, self.W3) + self.b3
        self.a3 = self.softmax(self.z3.flatten()) # Output probabilities

    def train(self, X, y, lr=0.015, momentum=0.9):
        # Backpropagation with momentum
        # ... training code with gradient descent ...
        return loss
```


HYBRID

python

 Copy  Download

```
# Lines 380-518 - Hybrid system combining all three
class HybridIntelligentSystem:
    """Hybrid system combining Fuzzy, GA, and NN"""
    def __init__(self, grid_size=20):
        self.fuzzy = FuzzyController() # Fuzzy component
        self.nn = NeuralNetwork()      # ANN component
        # GA is used externally to optimize fuzzy parameters

    def decide(self, controller_inputs, prev_dir, visited_positions=None,
               use_nn_weight=0.5, grid=None):
        # Get fuzzy scores
        fuzzy_scores, dirs = self.fuzzy.fuzzy_rules(...)

        # Get ANN predictions
        nn_input = controller_inputs.to_vector().reshape(1, -1)
        nn_probs = self.nn.forward(nn_input)

        # Combine fuzzy and ANN outputs
        combined = (1 - adaptive_weight) * fuzzy_normalized + adaptive_weight * nn_probs

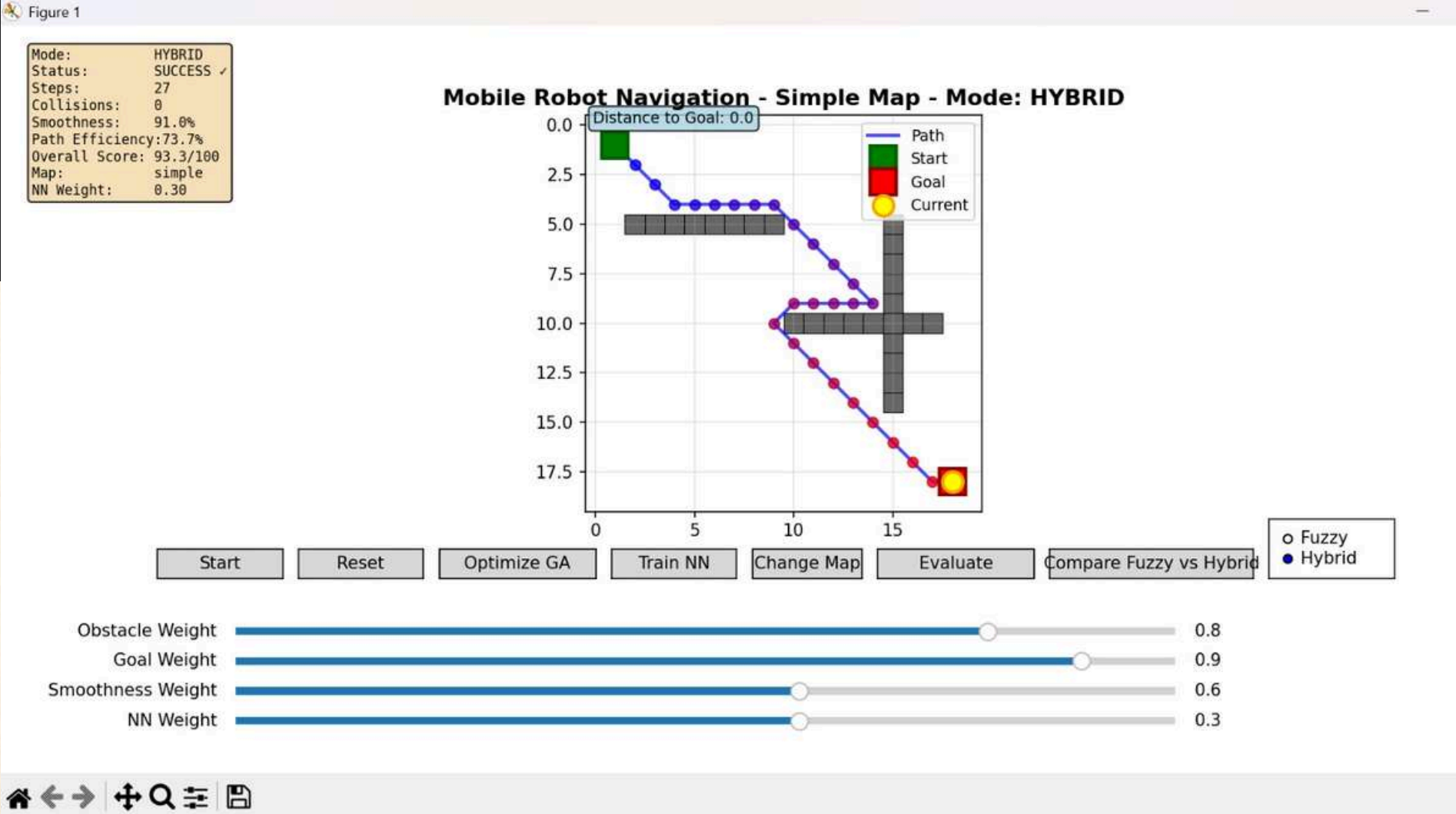
        # Make final decision
        best_idx = np.argmax(combined)
        return ControllerOutputs(direction, confidence, "hybrid")
```


Gui

```
# Create window and draw key elements
self.fig, self.ax = plt.subplots(figsize=(13, 10))

# Draw obstacles (gray blocks in our figures)
rect = patches.Rectangle((x,y), 1, 1, fc='dimgray', ec='black')

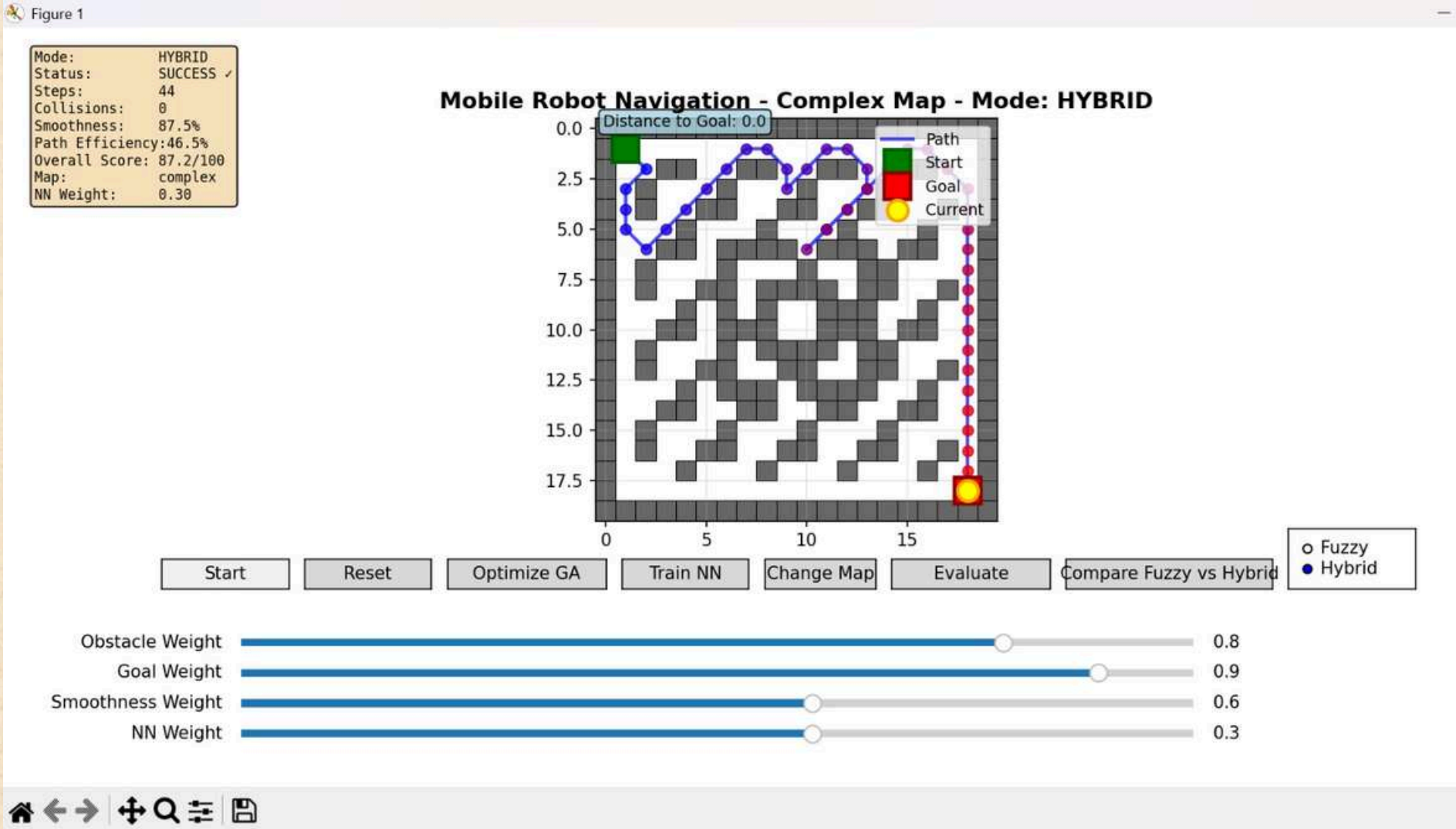
# Mark start (green) and goal (red)
ax.plot(start_x, start_y, 's', color='green', markersize=15)
ax.plot(goal_x, goal_y, 's', color='red', markersize=15)
```



Gui

```
self.btn_start = Button(plt.axes([0.10,0.26,0.08,0.04]), 'Start')
self.btn_reset = Button(plt.axes([0.19,0.26,0.08,0.04]), 'Reset')
self.btn_compare = Button(plt.axes([0.67,0.26,0.13,0.04]), 'Compare')

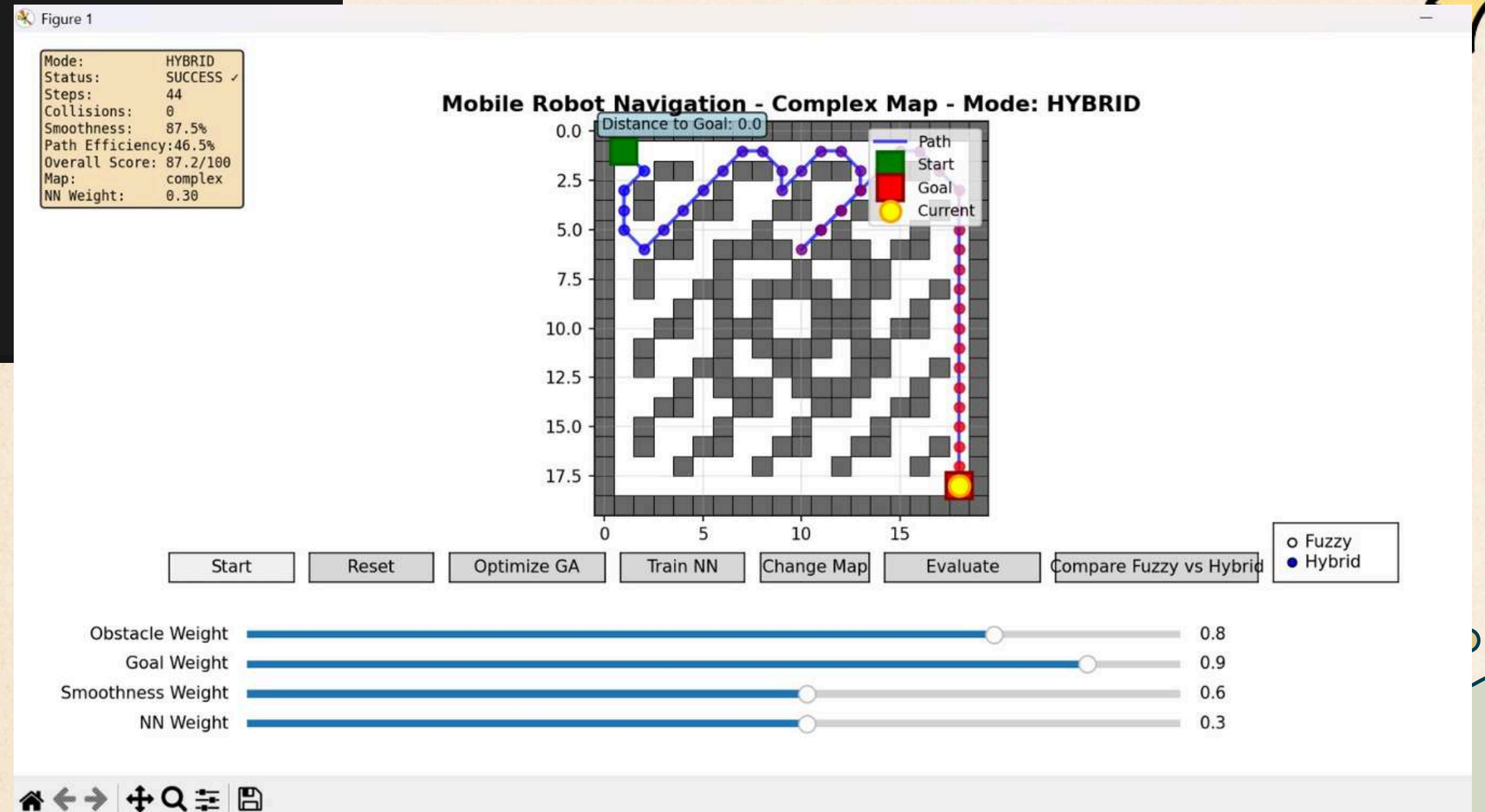
# Parameter sliders
self.sl_obs = Slider(plt.axes([0.15,0.18,0.60,0.02]), 'Obstacle Weight', 0, 1)
```



Gui

```
# Make it move and update
self.anim = FuncAnimation(self.fig, self.animate, interval=50)
```

```
def animate(self):
    self.step()      # Calculate next move
    self.draw()      # Redraw path
    self.update_stats() # Update live numbers
    plt.draw()       # Refresh screen
```



FORMULAS

Metric	Formula
Success Rate (%)	$\text{Success Rate} = \frac{\text{Number of successful trials}}{\text{Total trials}} \times 100$
Steps to Goal	$\text{Steps to Goal} = \text{Total steps taken during navigation}$
Collision Count	$\text{Collisions} = \text{Number of invalid movement attempts}$
Path Efficiency (%)	$\text{Path Efficiency} = \frac{\text{Optimal distance}}{\text{Actual distance traveled}} \times 100$
Path Smoothness (%)	$\text{Path Smoothness} = \left(1 - \frac{\text{Average turn angle}}{\pi}\right) \times 100$
Overall Performance (%)	$P = \min \left(100, \max(0, 40S + \min(20, E/5) + (20 - 2 \min(C, 10)) + Sm/10 + \max(0, 10 - St/50)) \right)$ S = Success (1 if goal reached, 0 otherwise), E = Path Efficiency (%), C = Collisions, Sm = Path Smoothness (%), St = Steps to Goal

RESULTS

Evaluation Using Fuzzy and Hybrid Mode on Simple Map

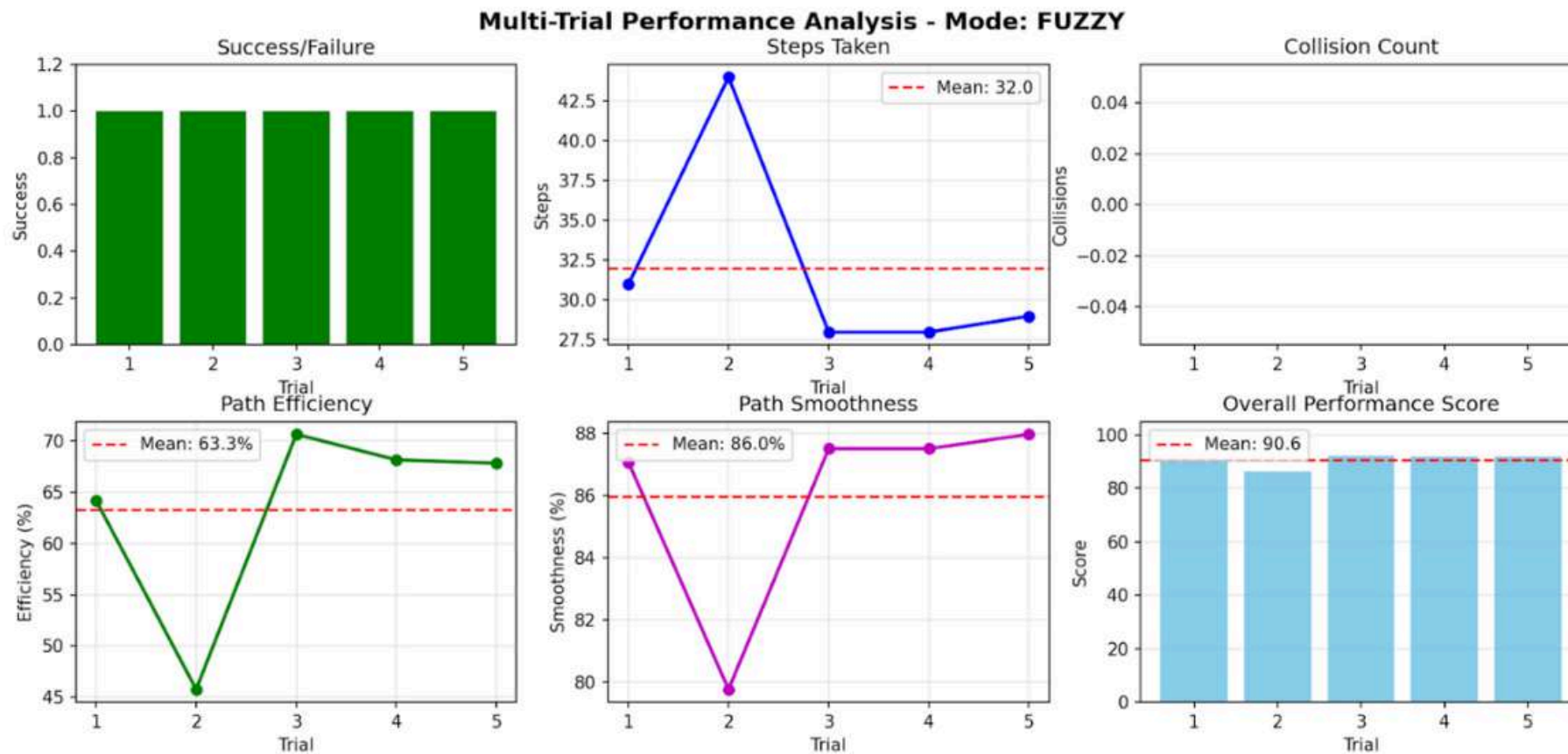


Figure 3: Evaluation of Navigation Controllers Using Fuzzy Mode on Simple Map

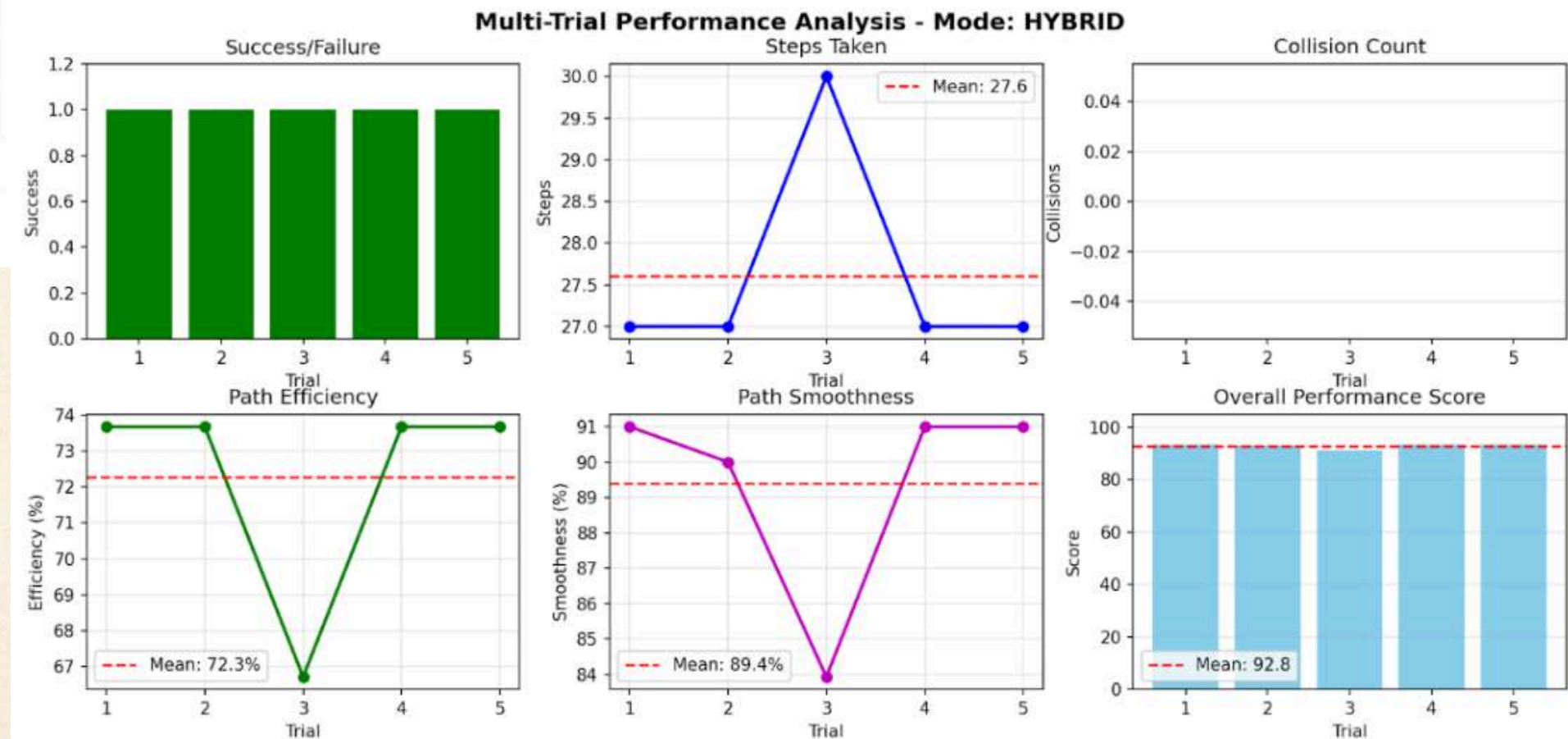


Figure 4: Evaluation of Navigation Controllers Using Hybrid Mode on Simple Map

RESULTS

Evaluation Using Fuzzy and Hybrid Mode on Complex Map

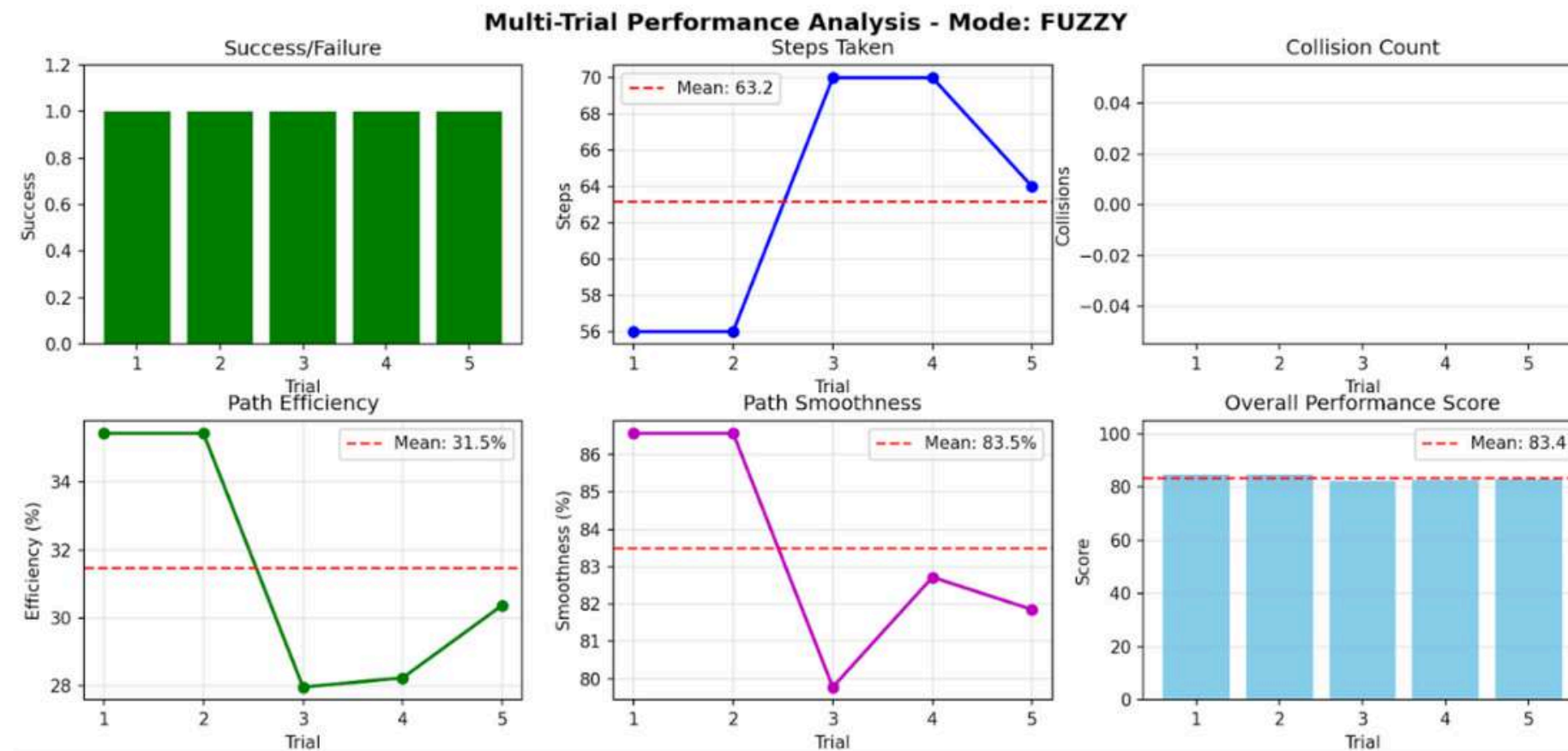


Figure 5: Evaluation of Navigation Controllers Using Fuzzy Mode on Complex Map

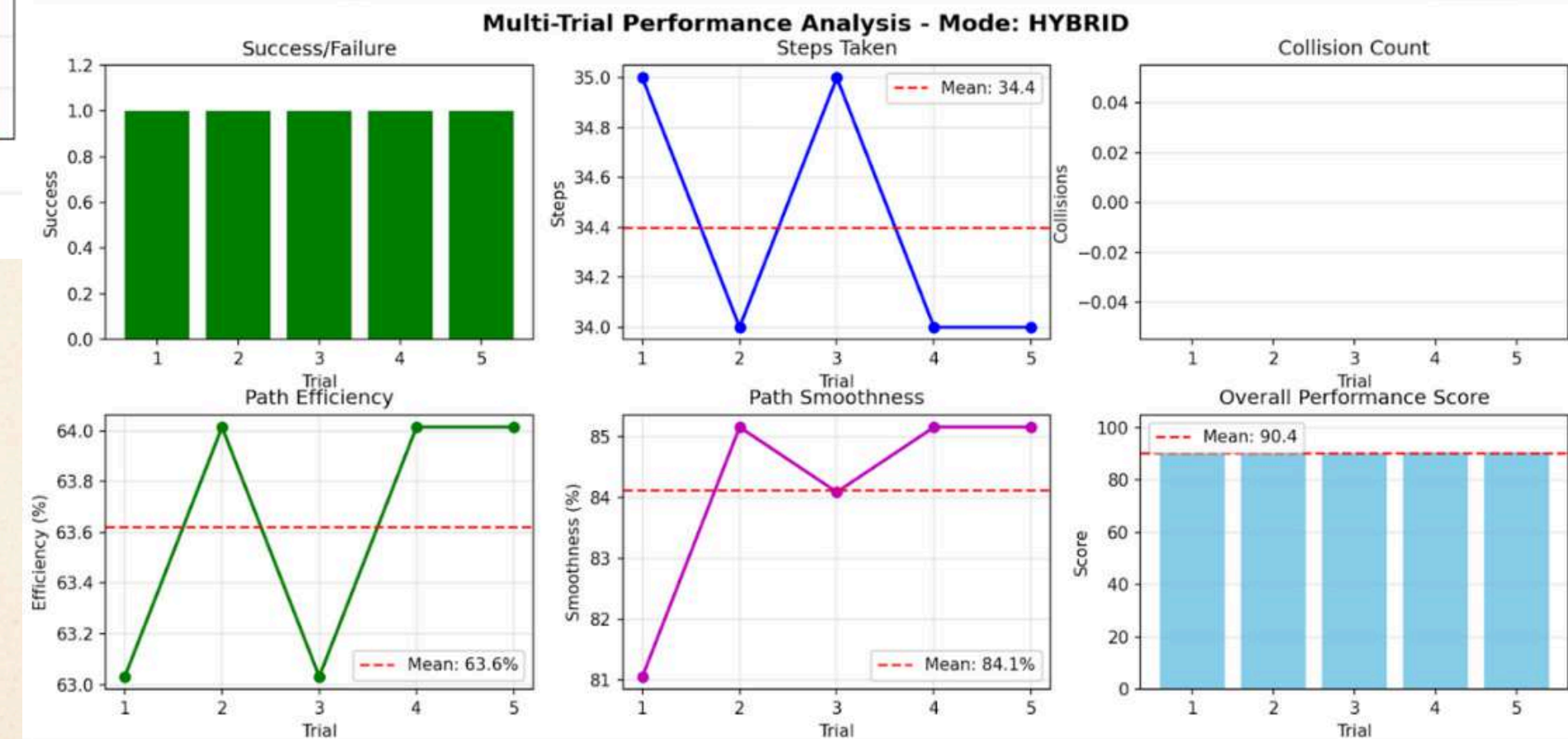
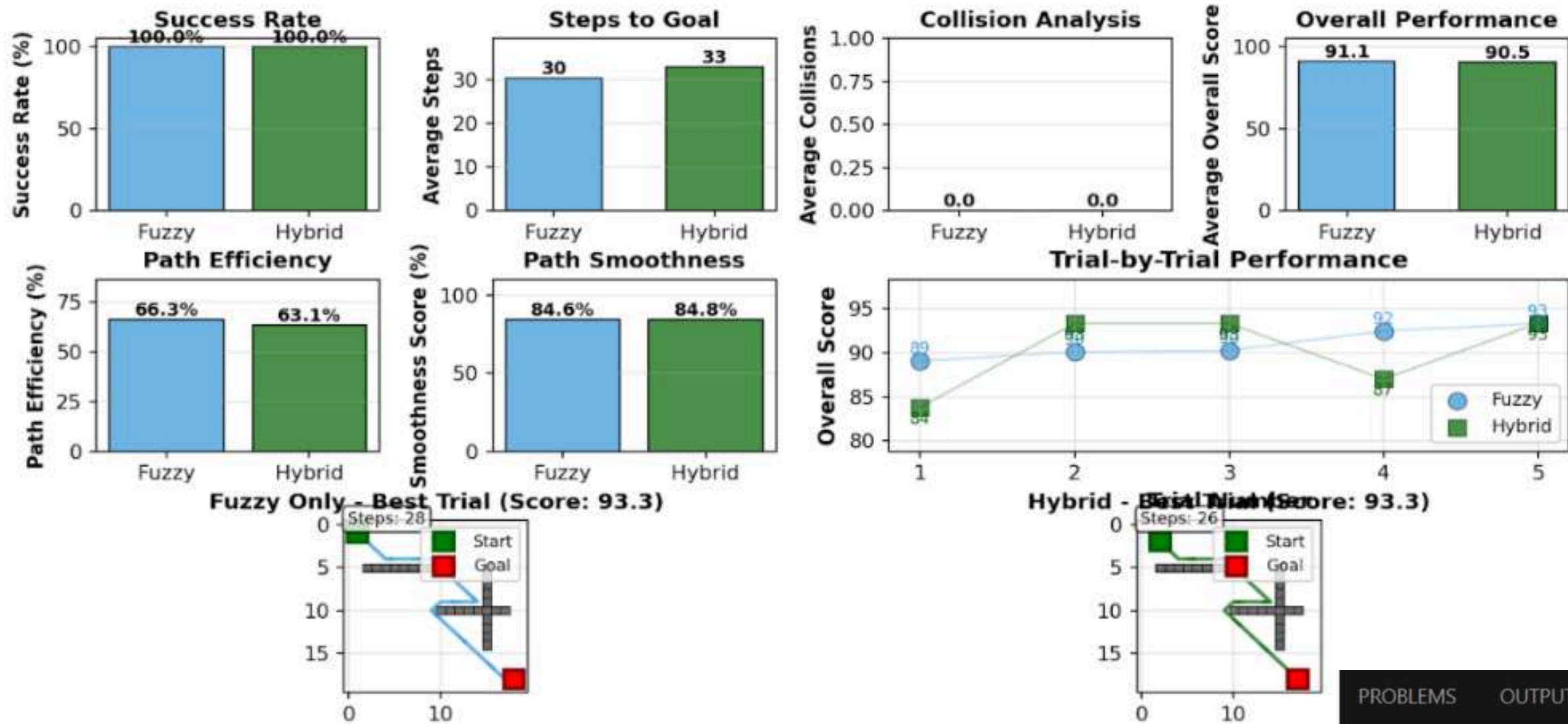


Figure 6: Evaluation of Navigation Controllers Using Hybrid Mode on Complex Map

RESULTS

FUZZY LOGIC vs HYBRID COMPARISON



Comparison of Fuzzy and Hybrid Mode on First Trial on Simple Map

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

=====

🏆 WINNER: FUZZY LOGIC ONLY

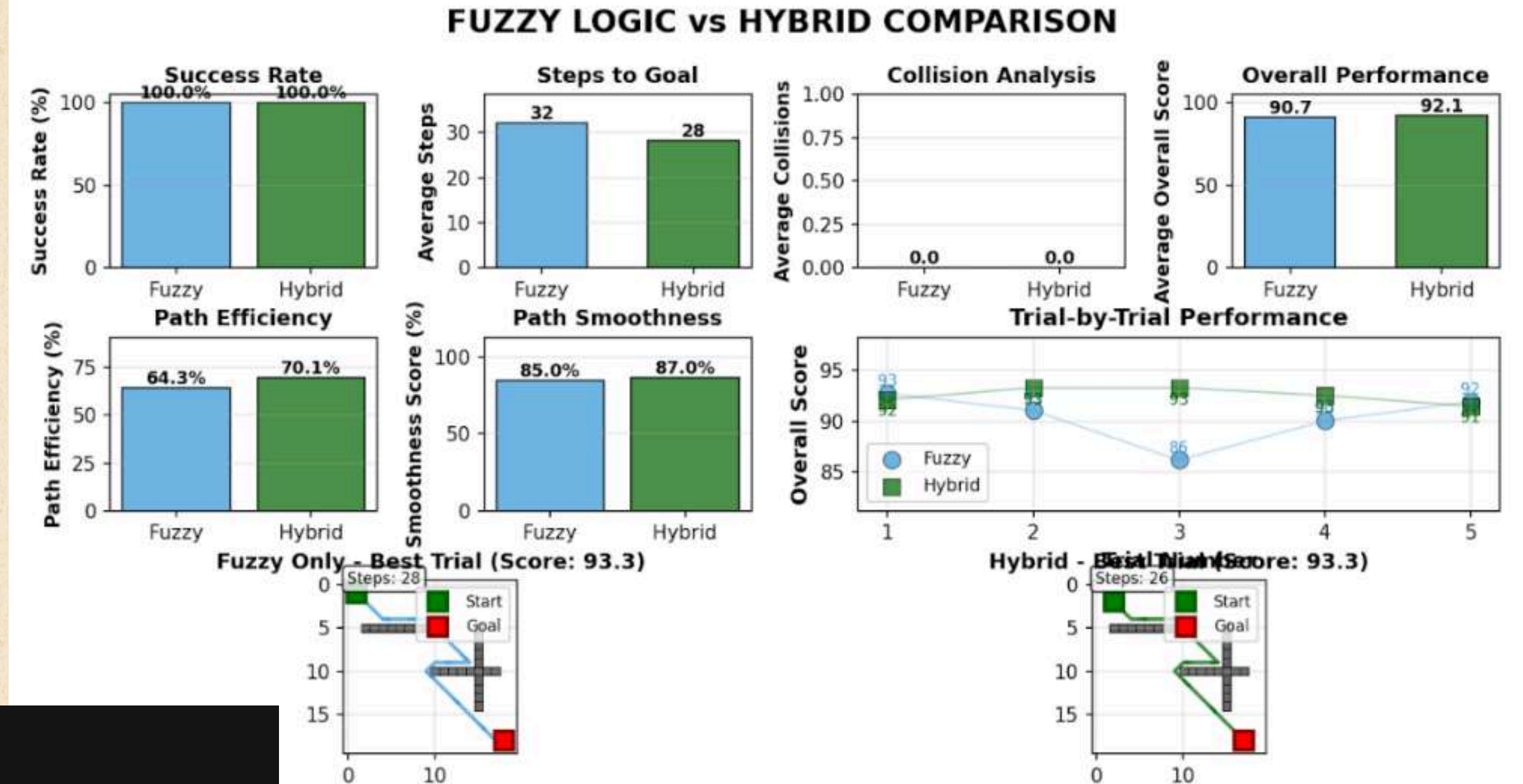
=====

📊 SUMMARY STATISTICS:

Fuzzy Only: 10/10 successes (91.1/100 avg)
Hybrid: 10/10 successes (90.5/100 avg)

RESULTS

Comparison of Fuzzy and Hybrid Mode on Second Trial on Simple Map



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

=====

🏆 WINNER: HYBRID (Fuzzy + GA + NN) by 1.6%

=====

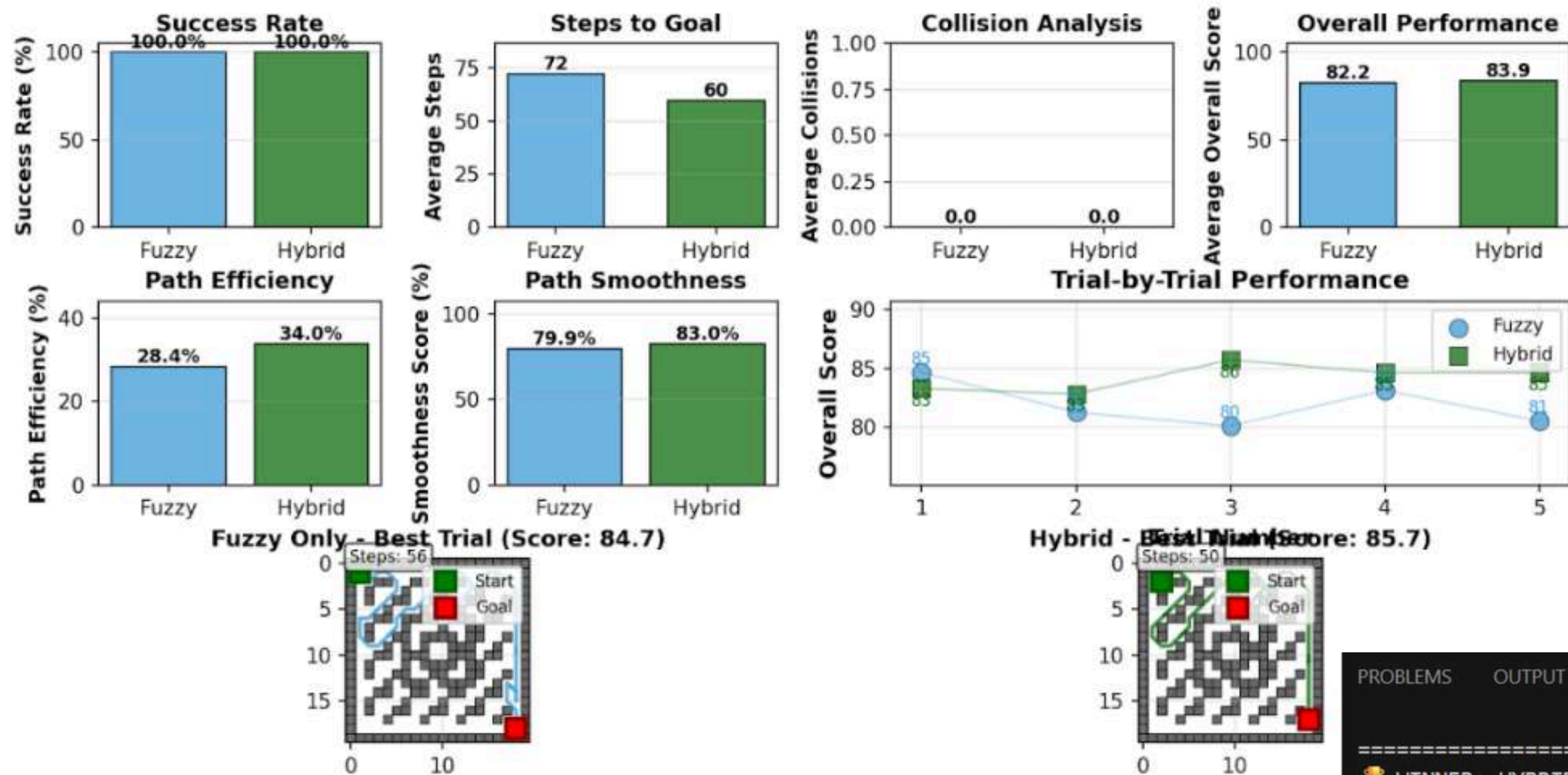
📄 SUMMARY STATISTICS:

Fuzzy Only: 10/10 successes (90.7/100 avg)

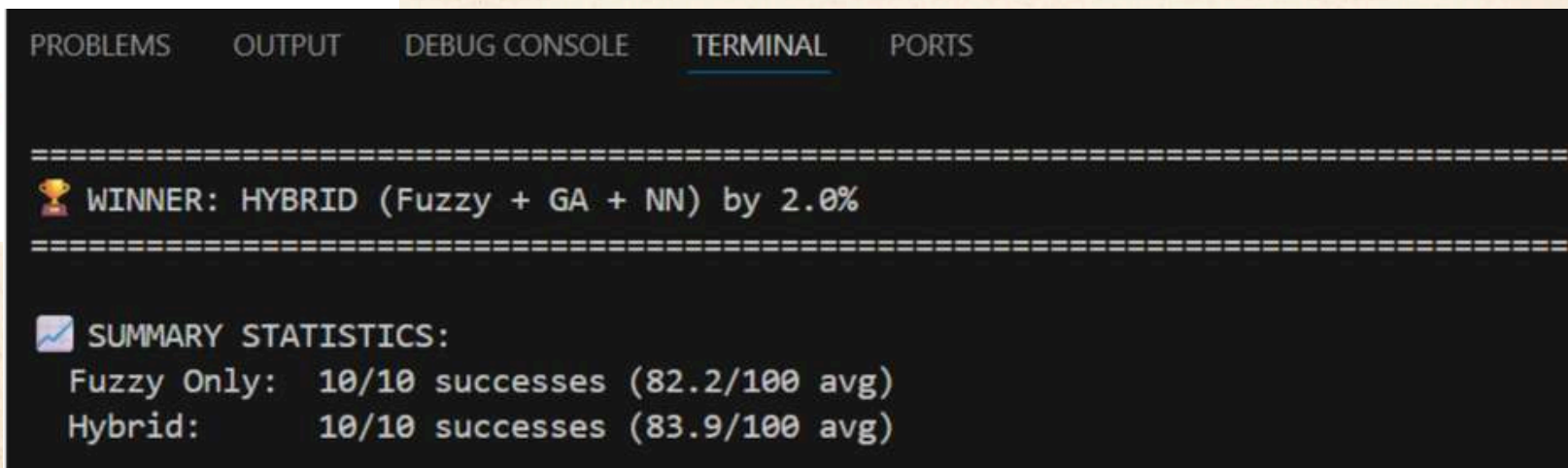
Hybrid: 10/10 successes (92.1/100 avg)

RESULTS

FUZZY LOGIC vs HYBRID COMPARISON



Comparison of Fuzzy and Hybrid Mode on Complex Map



ANALYSIS OF RESULTS

Standard Fuzzy Logic Controller

- Strong and consistent performance from the first trial
- Uses fixed, expert-defined rules
- No learning or adaptation phase required
- Achieved:
 - 100% goal-reaching success
 - High path efficiency
 - Smooth and stable navigation
- Best suited for stable environments and instant deployment

ANALYSIS OF RESULTS

Hybrid Fuzzy-GA-ANN Controller (Initial Trial)

- Operates using an adaptive learning framework
- First trial acts as a learning and exploration phase
- Slightly lower initial performance:
 - Longer path length
 - Lower efficiency (63.1%) compared to Fuzzy (66.3%)
- Performance still acceptable but not yet optimized

Hybrid Controller (After Learning)

- Performance improves significantly in subsequent trials
- Genetic Algorithm optimizes controller parameters
- Neural Network refines decision-making behavior
- Observed improvements:
 - Reduced steps to reach the goal
 - Increased path efficiency and smoothness
 - Higher overall performance score

CONCLUSION

- A hybrid computational intelligence controller for mobile robot navigation was successfully developed.
- The integration of Fuzzy Logic, Genetic Algorithm, and Neural Network improved decision-making and navigation performance.
- Simulation results show effective obstacle avoidance and efficient path planning in different environments.
- The hybrid approach is more robust and adaptable than individual soft computing techniques.
- This project demonstrates the potential of hybrid computational intelligence methods for autonomous robot navigation.



**THANK
YOU**